

- Heuristic Evaluation (Asking Experts)

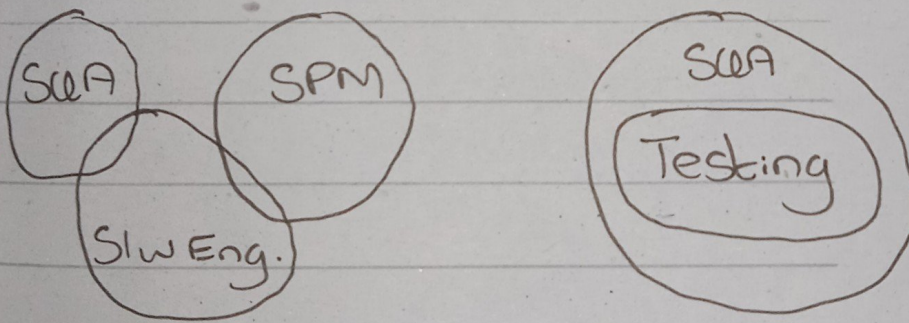
SQA - Course:

Software Quality is degree of conformance to Functional Req., Standards & Software Engineering Practices.

(Quality Control)

Testing:- Quality of Product / Improvement

SQA:- Steps that are being used to develop software



Software Quality Assurance: (Definition From IAPT)

Maintenance (Activity after Project is live)

Development (Activity before Project is live)

Week-9¹ (Session-2)

The main objective of Quality Assurance is to minimize the cost and enhance the Quality. It is known as Feasible Solution. Before testing, we define Acceptable level.

Characteristics of SQA (Next class)

(i) Contract

- Fixed scope
- per hr
- per resource

content: scope

timetable: Project Plan

(ii) contract with Supplier

SLAs (Service level Agreement)

(iii) Team work

(iv) coordination with internal & external team

(v) work continues when team members change

(vi) conduct Maintenance of software system

Types of Bugs

Error :- Mistake, Fault :- Functionality Error

Failure: when Fault runs it becomes Failure

Fault: (defect):

why Software Error Occurs

9 causes of Errors:-

(i) Faulty Requirement Definition:-

wrong definition of Requirement, misses to note a Function, incomplete definition of Requirement, Inclusion of unnecessary requirement.

(ii) client-developer communication Failure:-

• Misunderstanding of the client's requirement
have to do this slide

Next-slide lecture starting 8-slides.
Should read.

McCall's Classic Factor Model:-

- Product Operation
- Product Revision
- Product Transition

(i) Product Operation:-

- Correctness is related to Functionality, is about right output.
- Reliability is about Maintaining Service level, Affordable downtime, depends on business.
- Efficiency is about Managing the hardware Resources like processor, data storage, communication line bandwidth
- Integrity deals with the Software Security to prevent access to unauthorized persons.
- Usability

Product Revision:-

- Portability
- Reusability
- Interoperability
- Maintainability
- Flexibility
- Testability

Slide - 20 to start

- Corrective Maintenance: correction of Software Faults and Failures after Project is live
- Adaptive Maintenance: is the modification of Software to keep it usable in a changing environment.
- Perfective Maintenance (Enhancements and improvement of existing Software with respect to locally limited Issues.

Maintainability is related to corrective Maintenance.

Flexibility is related to adaptive maintenance and Perfective Maintenance.

Testability requirements deal with the testing of an information System as well as with its operation.

$2+2 = 4$ (Pre-defined Result)

$(2+2) \times 4$

$2+2 = 4$ (Pre-defined Intermediate Result)

Slide-7 Quiz-1 Quiz-2 (Slide-8 onwards)
(week-4 After topics)

Product Transition:-

- Portability :- Compatability (Architecture & OS)
- Reusability :- Reuse of code
- Interoperability :- communicating with different systems

In Software development, defining clear requirements is important not only for client but also for developer. While the client focuses on what the software should do, the developer may include

week-5:-

Testing:

- To ensure Functional and non-Functional requirements are working.

Start From Slide-6

Types OF Testing.

Static Testing:- Code is not executed

Dynamic " :- code is being executed

Reviewing Documents/ code (Static)

Executing a login Feature to check if it accepts valid credentials and reject invalid ones.

Test data is dummy data.

Retesting:- New change in Functionality is being verified by retesting. Focuses on the change is rightly being implemented.

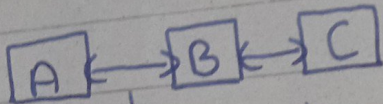
Regression:- unchange code / Functionality is being working correctly

was absent and have to take notes of this lecture From Slide-6.

Integration Testing:-

There are two types;

- (i) Unit / Module / component Integration → Developer Team
- (ii) System Int. Testing → QA / Tester



connection testing is component Int. Test

System testing is done to check both the Functional and non-Functional Requirements of the System.

The thing we are doing for testing is Test obj Process

Acceptance testing's purpose is to provide end-users with confidence that the system will function acc to their expectations.

Types:-

- (i) User Acceptance Testing
- (ii) Operational " "
- (iii) Contract and Regulation acceptance Testing

Stages / Levels:-

- (i) Alpha Testing: takes place at developer's site
- (ii) Beta Testing: " " " customer's "

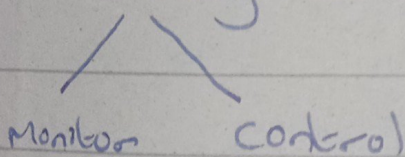
Test - Types:-

- (i) Functional
 - (ii) Non-Functional
 - (iii) Structural
 - (iv) Test after code changes
-

Week-5 : S2 CLO-1 & CLO-3 ^{can be} (Both) may
Fundamental Test Process:-

Process → Steps to develop a product / software

Planning → Task, Resource, Time



We will monitor the execution. If it fails, the correction is control.

Analysis & Design → design test cases

Implementation & Execution → Do Testing

week-5 so2 slide 7 start

Test Data

Test object $\rightarrow$ Application under Test

Test bases

Test condition $\rightarrow$ Addition

Data	cond	Data	Expected outcome	
2	+	2	= 4	$\rightarrow$ Test-case

Test Environment

Pre-condition: state of System before Test

Post-condition: " " " After Test

(CLO-2 Starting):-

Test-condition is some characteristic of our software that we can check with a set test

Test-Procedure:-

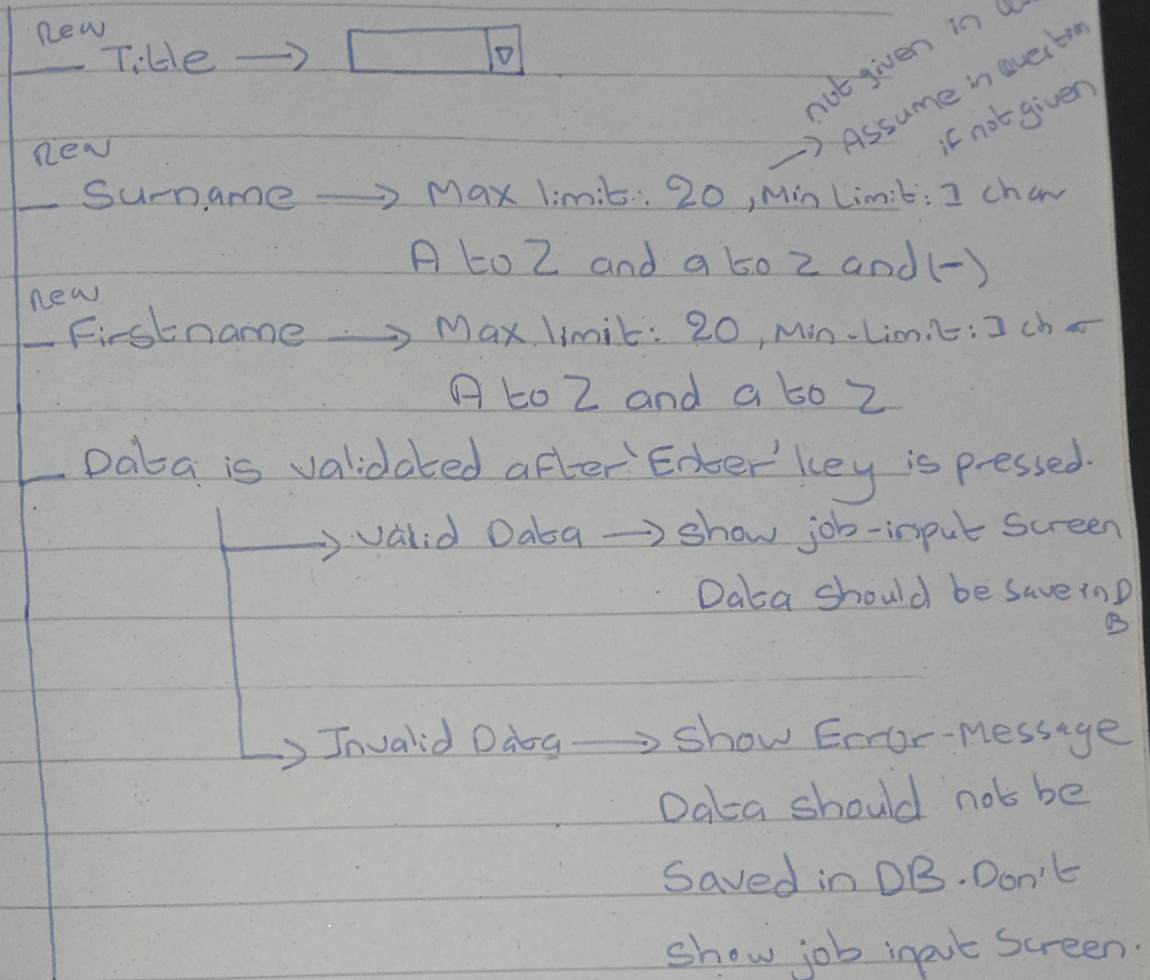
A sequence of actions to execute a test

Three Steps:-

(i) decide on test condition
two more.

Test-case Designing:- (Slide-9 Question)

Form:-



Test object: Form

Test-condition:

Required & Non-Req.

- Max Limit } Surname
- Min Limit }

- Max Limit } First Name
- Min Limit }

- Allowed & not Allowed First name

- " " " Surname

week-6 : slide-3

Test-Procedure slide-12

Black-box testing involves the testing not going depth, testing UI and use its Specification. including Functional and non-Functional req.

white-box is testing the code is optimize or not. It is about code structure

Black-box testing is enough For testing the Software that its meets the user req. or not.

CLO-2 (Black-box Techniques)-

- (i) Equivalence Partitioning
- (ii) Boundary Value Analysis
- (iii) Decision Tables
- (iv) State Transition

~~RT~~

Equivalence Testing:-

Boundary Value Analysis:-

Data $\xrightarrow{\text{i/P}}$ Partitions $\rightarrow$ Similar O/P

It is the software testing technique that divides the input data into partitions

$$D = \{1, 2, 3, \dots, \infty\}$$

$$\{1, 2, \dots, 10\} = x \rightarrow \text{O/P}$$

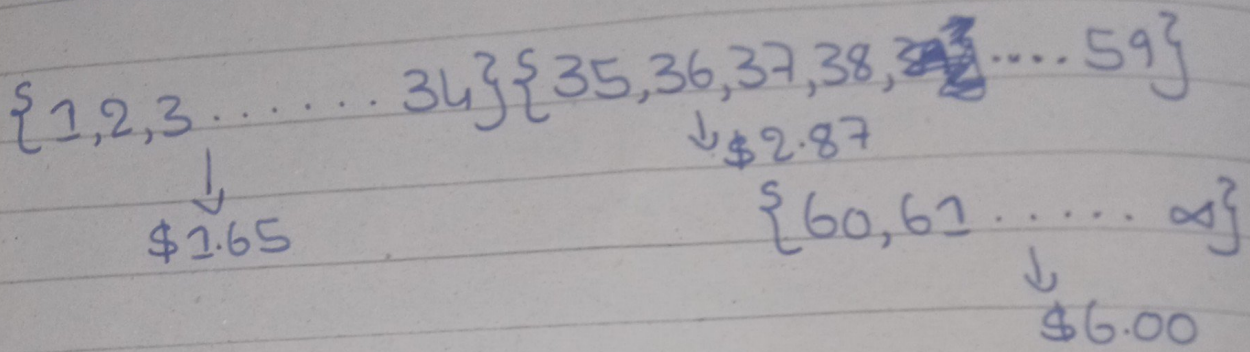
$$\{11, 12, \dots, 20\} = y \rightarrow \text{O/P}$$

~~If~~ After Partition, if we test from boundary values so it will be boundary value analysis. If we test through the random values inside the boundary values are Equivalence, it may be boundary values as well.

Let's take an example of Month

$$\begin{array}{cccc} \{-\infty, \dots, -1\} & \{0\} & \{1, 2, \dots, 12\} & \{13, \dots, \infty\} \\ \downarrow & \downarrow & \downarrow & \downarrow \\ \text{Error} & \text{Error} & \text{Accept} & \text{Error} \end{array}$$

Partition For Example of Life Insurance.



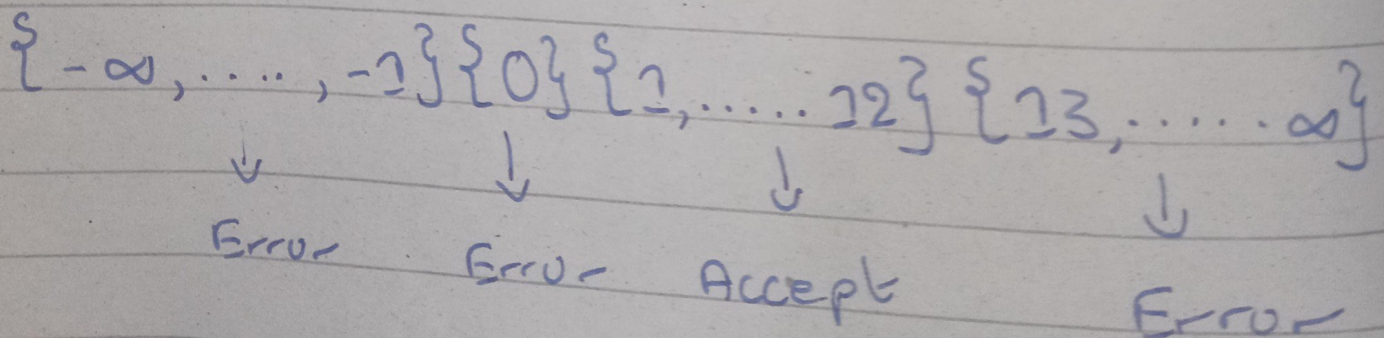
Example From Image.

Testcases

S.No	Equivalence Partition	Test-Data	Expected Result
1	under 35	3, 5	2.15
2	Between 35 & 39	40	3.37
3	Above 60	73	6.5
4	Negative Age	-66	Invalid Input
5	Age as 0	0	Invalid Input

Boundary Value Analysis:-

Taking Month Example



e.

Left Neighbour

N/A

 $\{-\infty, \dots, -1\}$ $\{0\}$ $\{1, \dots, 12\}$

Partition

 $\{-\infty, \dots, -1\}$ $\{0\}$ $\{1, \dots, 12\}$ $\{13, \dots, \infty\}$

Right

 $\{0\}$ $\{1, \dots, 12\}$ $\{13, \dots, \infty\}$

N/A

Test

 $-1, 0$ $-1, 0, 1$ $0, 1, 12, 13$ $12, 13$

No. of Cases

2

3

4

2

Test - Cases

sult

Now take the Example of Slide

$\{-\infty, \dots, -1\}$ $\{0\}$ $\{1, \dots, 34\}$ $\{35, \dots, 59\}$ $\{60, \dots, 999\}$ $\{1000, \dots, \infty\}$

t

t

Left

N/A

 $\{-\infty, \dots, -1\}$ $\{0\}$ $\{1, \dots, 34\}$ $\{35, \dots, 59\}$ $\{60, \dots, 999\}$ $\{1000, \dots, \infty\}$

Partition

 $\{-\infty, \dots, -1\}$ $\{0\}$ $\{1, \dots, 34\}$ $\{35, \dots, 59\}$ $\{60, \dots, 999\}$ $\{1000, \dots, \infty\}$

Right

 $\{0\}$ $\{1, \dots, 34\}$ $\{35, \dots, 59\}$ $\{60, \dots, 999\}$ $\{1000, \dots, \infty\}$

N/A

Test

 $-1, 0$ $-1, 0, 1$ $0, 1, 34, 35$ $34, 35, 59, 60$ $60, 999, 59, 1000$ $999, 1000$

No. of Cases

2

3

4

4

4

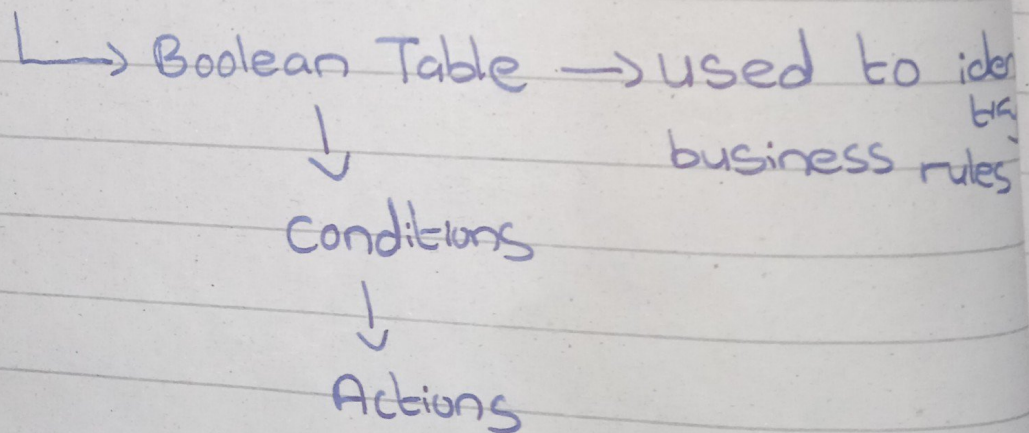
2

Test-cases

S.No	Partition	Test-Data	Expected Result
1	under 35	0, 1, 34, 35	2.15
2	Between 35 & 39	34, 35, 59, 60	3.37
3	Above 60	59, 60, 999, 1000	6.5
4	Negative Age	-1, 0	Invalid
5	Age as 0	-1, 0, 1	Invalid

S.No	Partition	Test-Data	Expected
1	under 35	0	2.15
2	"	1	3.37 2.15
3	"	34	2.15
4	"	35	2.15
5	Between 35 & 39	34	

Decision Tree:-



Total no. of business Rules = $2^{\text{Total No. of Conditions}}$

Decision Tree is a binary tree. Each node has 2 Paths either true or False.

Nodes $\rightarrow$ conditions

Leaves $\rightarrow$ Actions

Path $\rightarrow$ Business rule

Example:-

A customer request a cash withdrawal. One of the business rule of ATM is that the ATM machine pays out if sufficient funds in their Amount or if credit granted.

Solution:-

Conditions:-

$\rightarrow$ Independent
(comes First in Table)

(i) Has sufficient Funds

(ii) Is credit granted $\rightarrow$ Dependent

Action:-

(i) Allow cash withdrawal

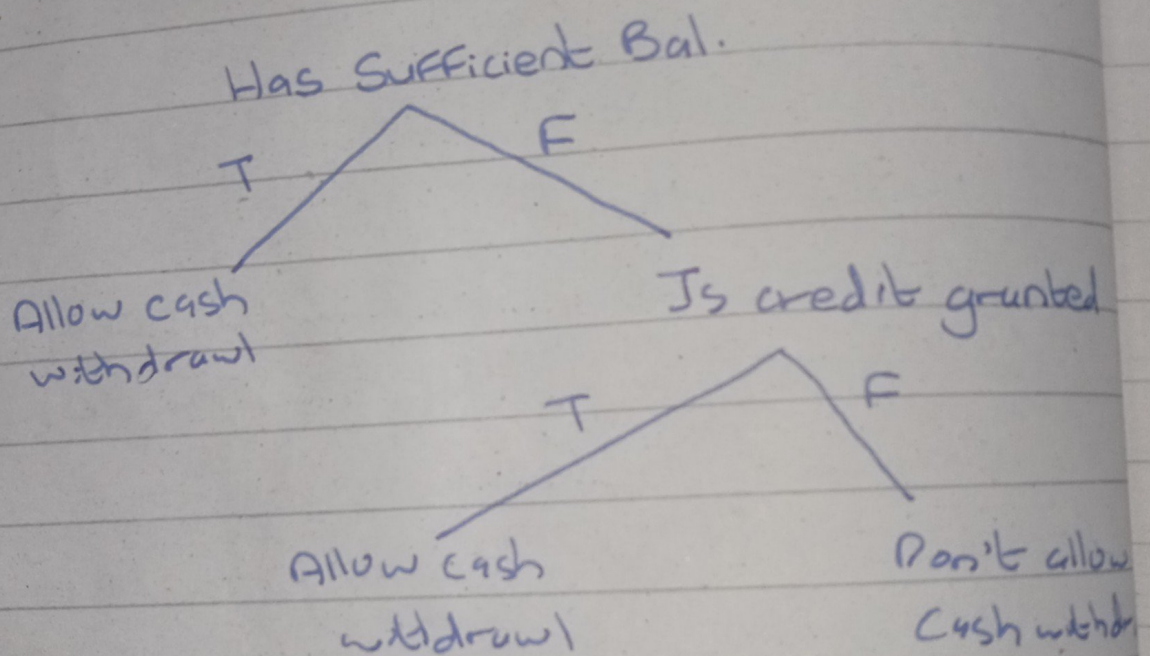
Step -1:-

	R ₁	R ₂	R ₃	R ₄
Conditions				
Has sufficient balance?	F	F	T	T
Is credit granted?	F	T	F	T
Action				
Allow cash withdrawal	F	T	T	T

Step-2: Add don't care (X) where the output depends on particular not all inputs.

Step-3: Remove Redundancy in Table.

Decision-Tree:-



Example-2:- (From Image)

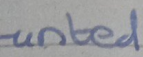
the output
all inputs

1

unted

allow

withdraw



allow

withdraw

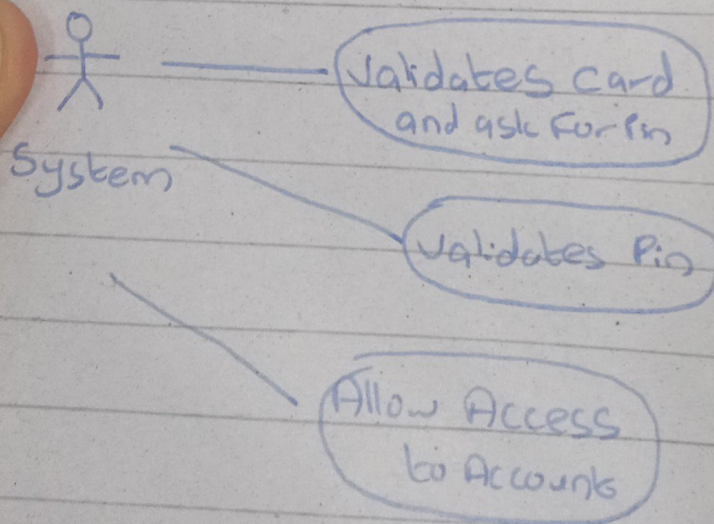
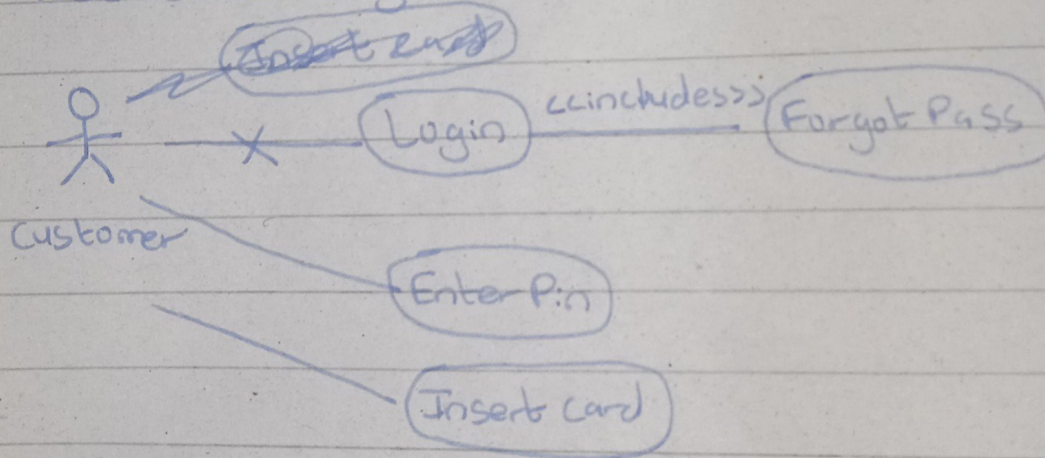
Use-case Testing:-

what actions are performed by user.

⌚ (Actor) ○ (Action)

(Extend → Alternative Flow) (Include is another Independent Function related)

Use-case Diagram:-



Mid-Paper Pattern

CLO-1

Theory

week 1 S1

week 1 S2

week 2 S1 & S2

week 3 S1 & S2

week 5 S1

CLO-2

Text-ans

week 5 S2

week 6 S2

week 7 S1

week 7 S2

week 8 S1

week 8 S2